

Reviewer Pack — Valuation Rank of Polynomial Coefficients Bounds ACC^0 Circuit Size

Entry 593ecfc3498f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 01:41:34 UTC

Valuation Rank of Polynomial Coefficients Bounds ACC^0 Circuit Size

Entry ID: 593ecfc3498f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 01:41:34 UTC

1. Conjecture

Field A (mathematical branch): Valuation Theory **Field B** (complexity object): ACC^0 Circuit Size

Statement:

For any CNF formula Φ with n variables, the number of distinct p -adic valuations of its clause-encoding polynomial's coefficients is $\Theta(\log n)$ iff Φ has an ACC^0 circuit of size $O(n^3)$.

Rationale (proposer's reasoning):

Valuations capture arithmetic structure that may correlate with circuit complexity. The p -adic valuation's rank could expose algebraic constraints on polynomial representations of Boolean functions, linking valuation theory to circuit size bounds.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 585c8fc7e0782033

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def generate_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = set(random.randint(1, n) for _ in range(3))
        cnf.append(clause)
    return cnf

def polynomial_encoding(cnf):
    n = max(max(clause) for clause in cnf)
    variables = [0] * n
    poly = 1
    p = 2 # Using p-adic valuation over Q_2 (binary)

    for clause in cnf:
        term = 1
        for v in clause:
            if random.choice([True, False]):
                term *= variables[v - 1]
            else:
                term *= (1 - variables[v - 1])
        poly += term

    return p, poly

def p_adic_valuation(poly, p):
    valuation = 0
    while poly % p == 0 and poly != 0:
        poly //= p
        valuation += 1
    return valuation

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        cnf = generate_cnf(n, 2 * n)
        p, poly = polynomial_encoding(cnf)
```

```

valuations = [p_adic_valuation(coeff, p) for coeff in poly.coefficients()]
distinct_valuations = len(set(valuations))

instances_tested = len(valuations)
conjecture_holds = False
counterexample = ""

if n <= 40:
    expected_valuation_rank = math.log(n, 2)
    expected_circuit_size = n**3 * 10**6 # Simplified upper bound for ACC^0 circuit size

    if abs(distinct_valuations - expected_valuation_rank) < 1 and poly.size() <= expected_circuit_size:
        conjecture_holds = True
    else:
        counterexample = f"n={n}, distinct_valuations={distinct_valuations}, expected_valuation_rank={expected_valuation_rank}"

results.append({
    "metric_name": "valuation_rank",
    "metric_value": distinct_valuations,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
})

return {
    "seed": seed,
    "results": results
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    all_results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        all_results.append(result)

    total_valuation_rank = sum(sum(trial["results"][0]["metric_value"] for trial in results) for results in all_results)
    num_trials = len(all_results)
    avg_valuation_rank = total_valuation_rank / num_trials

    support_fraction = sum(any(trial["results"][0]["conjecture_holds"] for trial in results) for results in all_results) / num_trials

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_valuation_rank:.2f} std=... support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, results in enumerate(all_results, start=1) if not any(trial["results"][0]["conjecture_holds"] for trial in results))
        print(f"RESULT: FALSIFIED counterexample=\"valuation_rank_mismatch\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe5b1309.py", line 93, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe5b1309.py", line 58, in run_trial
    valuations = [p_adic_valuation(coeff, p) for coeff in poly.coefficients()]
                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'int' object has no attribute 'coefficients'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to AttributeError, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Fix the test code to handle integer coefficients properly and re-run experiments with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	110271
2	propose	ollama_remote	qwen3:8b	0	0	99457
3	preregistration	ollama_remote	qwen3:8b	0	0	23977
4	novelty	ollama_remote	qwen3:8b	0	0	20643
5	novelty	ollama_remote	qwen3:8b	0	0	18334
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12463
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9718
8	judge	ollama_remote	qwen3:8b	0	0	15168

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 310031 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in <research/audit/593ecfc3498f.jsonl>)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in <research/audit/593ecfc3498f.jsonl> for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/593ecfc3498f.tar.gz` (if generated)